

🌟 DAY - 36 : Master JavaScript Performance — Debouncing, Throttling, Memoization Explained



JS JAVASCRIPT

PERFORMANCE OPTIMIZATION

LEARNING DAY 36

TOPICS

DEBOUNCING | THROTTLING | MEMOIZATION

🚀 LEVEL

ADVANCED JAVASCRIPT

Created by: Neeraj | [LinkedIn: neeraj-kumar1904](#) 📁 | [X: @_19_neeraj](#) 🐦 | [GitHub: Neeraj05042001](#) 🧑

🎯 What is Performance?

Performance in JavaScript refers to how efficiently and quickly your code executes, uses memory, and responds to user interactions, and how optimally it uses system resources like memory and CPU.

Good performance means your web application loads quickly, responds smoothly to user inputs, and doesn't consume excessive memory or processing power. It encompasses:

- ⚡ **Execution Speed:** How fast your code runs
- 💬 **Memory Usage:** How efficiently memory is allocated and freed
- 🖱️ **Responsiveness:** How quickly the UI responds to user actions
- 📊 **Resource Utilization:** How well your code uses CPU, network, and other resources

🔍 Knowledge Check

- Q: What are the main aspects of JavaScript performance?

🎯 Why Performance is Important?

Performance directly impacts:

👤 User Experience

- **Fast Loading:** Users expect pages to load within 2-3 seconds
- **Smooth Interactions:** Laggy interfaces frustrate users
- **Battery Life:** Poor performance drains mobile device batteries

Business Impact

- **Conversion Rates:** 100ms delay can reduce conversions by 7%
- **SEO Rankings:** Google considers page speed for search rankings
- **User Retention:** Slow apps have higher bounce rates

Technical Benefits

- **Scalability:** Efficient code handles more users
- **Cost Reduction:** Less server resources needed
- **Maintainability:** Optimized code is often cleaner

Knowledge Check

- ▶ Q: How does a 100ms delay affect business metrics?

Important Performance Techniques

1. Debouncing - The Patient Wait

Debouncing is a technique that delays the execution of a function until a specified time has passed since the last time it was invoked. Think of it as waiting for someone to finish talking before you respond.

How Debouncing Works:

- When an event occurs repeatedly, debouncing ensures the function executes only after the user stops triggering the event for a given period.
- It **resets the timer** each time the event is triggered.
- Perfect for scenarios where you only care about the final state.

Common Use Cases:

- **Search Input:** Wait for user to stop typing before making API calls
- **Form Validation:** Validate only after user finishes entering data
- **Auto-save:** Save document only after user stops editing
- **Resize Events:** Update layout only after window resize is complete
- **API calls triggered by user input**

Implementation:

```
function debounce(func, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => func.apply(this, args), delay);
  };
}

// Example: Search functionality
const searchInput = document.getElementById('search');
const debouncedSearch = debounce((query) => {
  console.log(`Searching for: ${query}`);
  // API call here
}, 300);

searchInput.addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});
```

Knowledge Check

- Q: What happens if a user types rapidly in a debounced search field?
- Q: When would you use debouncing over throttling?

2. Throttling - The Rate Limiter

Throttling limits how often a function can be executed, ensuring it runs at most once per specified time interval, regardless of how many times it's called. It's like taking photos at regular intervals during a concert, regardless of how exciting it gets.

How Throttling Works:

- Executes the function immediately on first call
- Ignores subsequent calls until the time limit has passed
- Maintains a consistent execution rate
- Perfect for scenarios requiring regular updates

Common Use Cases:

- **Scroll Events:** Update scroll position indicators
- **Mouse Movement:** Track cursor position for games or animations
- **Button Clicks:** Prevent rapid-fire submissions
- **API Rate Limiting:** Ensure you don't exceed API call limits

- **Real-time Updates:** Send periodic updates to servers

💡 Implementation:

```
function throttle(func, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      func.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

```
// Example: Scroll tracking
const throttledScroll = throttle(() => {
  console.log('Scroll position:', window.scrollY);
}, 100);

window.addEventListener('scroll', throttledScroll);
```

🔄 Debouncing vs Throttling:

Aspect	Debouncing	Throttling
Execution	After delay ends	At regular intervals
Use Case	Search, resize	Scroll, mouse move
Behavior	Waits for pause	Executes periodically

🔍 Knowledge Check

- ▶ Q: In throttling with a 100ms limit, how many times will a function execute if called 10 times in 500ms?
- ▶ Q: What's the key difference between debouncing and throttling?

3. 🧠 Memoization - The Smart Cache

Definition: Memoization is a caching technique that stores function results to avoid redundant calculations for the same inputs.

It's like remembering the answer to a math problem you've already solved.

- Stores function results using input parameters as cache keys
- On subsequent calls with same parameters, returns cached result instantly
- Trades memory for computational speed
- Most effective for functions with overlapping subproblems

🎯 When to Use Memoization:

- **Recursive Functions:** Fibonacci, factorial calculations
- **Expensive Computations:** Complex mathematical operations
- **Repeated API Calls:** Cache API responses for same requests
- **Dynamic Programming:** Solving optimization problems
- **Pure Functions:** Functions that always return same output for same input

💡 Implementation:

```
// Simple memoization
function memoize(fn) {
  const cache = {};
  return function(...args) {
    const key = JSON.stringify(args);
    if (key in cache) {
      console.log('Cache hit!');
      return cache[key];
    }
    console.log('Computing...');
    const result = fn.apply(this, args);
    cache[key] = result;
    return result;
  };
}

// Example: Expensive calculation
const expensiveOperation = memoize((n) => {
  let result = 0;
  for (let i = 0; i < n * 1000000; i++) {
    result += i;
  }
  return result;
});

console.log(expensiveOperation(100)); // Computing...
console.log(expensiveOperation(100)); // Cache hit!
```

✅ Example 2:

```
function memoize(fn) {
  const cache = new Map();
```

```

return function(...args) {
  const key = JSON.stringify(args);
  if (cache.has(key)) {
    console.log('Cache hit!');
    return cache.get(key);
  }
  const result = fn(...args);
  cache.set(key, result);
  return result;
};
}

// Example: Memoized Fibonacci
const fibonacci = memoize((n) => {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
});

console.log(fibonacci(40)); // Fast execution due to memoization

```

Advanced Memoization with LRU Cache:

```

class LRUCache {
  constructor(maxSize = 100) {
    this.maxSize = maxSize;
    this.cache = new Map();
  }

  get(key) {
    if (this.cache.has(key)) {
      const value = this.cache.get(key);
      this.cache.delete(key);
      this.cache.set(key, value); // Move to end
      return value;
    }
    return undefined;
  }

  set(key, value) {
    if (this.cache.has(key)) {
      this.cache.delete(key);
    } else if (this.cache.size >= this.maxSize) {
      const firstKey = this.cache.keys().next().value;
      this.cache.delete(firstKey);
    }
    this.cache.set(key, value);
  }
}

```

Knowledge Check

- ▶ Q: What's the main benefit of memoization?
- ▶ Q: What are the trade-offs of using memoization?

Memory Leaks - The Silent Performance Killer

A **memory leak** occurs when your application allocates memory but fails to release it when it's no longer needed, causing memory usage to grow continuously

Common Causes:

1. Global Variables

```
// ❌ Memory Leak
function createLeak() {
  leakedVar = "I'm global!"; // Missing var/let/const
}

// ✅ Fix
function noLeak() {
  let localVar = "I'm scoped!";
}
```

2. Event Listeners Not Removed

```
// ❌ Memory leak
function attachListener() {
  const button = document.getElementById('myButton');
  button.addEventListener('click', handleClick);
  // Button reference and listener not cleaned up
}

// ✅ Proper cleanup
function attachListener() {
  const button = document.getElementById('myButton');
  const handleClick = () => console.log('clicked');

  button.addEventListener('click', handleClick);

  // Cleanup when done
  return () => {
    button.removeEventListener('click', handleClick);
  };
}
```

3. Closures Holding References

```
// ❌ Memory leak
function createHandler() {
  const largeData = new Array(1000000).fill('data');

  return function() {
    // Even though largeData isn't used,
    // closure keeps it in memory
    console.log('Handler called');
  };
}

// ✅ Better approach
function createHandler() {
  // Only keep what you need
  return function() {
    console.log('Handler called');
  };
}
```

4. Timers Not Cleared

```
// ❌ Memory leak
function startTimer() {
  setInterval(() => {
    console.log('Timer running');
  }, 1000);
  // Timer keeps running forever
}

// ✅ Proper cleanup
function startTimer() {
  const intervalId = setInterval(() => {
    console.log('Timer running');
  }, 1000);

  // Clear when needed
  return () => clearInterval(intervalId);
}
```

Memory Leak Detection:

1. Chrome DevTools Memory Tab
2. Performance Timeline for memory usage patterns
3. Heap Snapshots to identify growing objects
4. Memory usage monitoring in production


```
// Monitor memory usage
function monitorMemory() {
  if (performance.memory) {
    console.log({
      used: Math.round(performance.memory.usedJSHeapSize / 1048576) + ' MB',
      total: Math.round(performance.memory.totalJSHeapSize / 1048576) + ' MB',
      limit: Math.round(performance.memory.jsHeapSizeLimit / 1048576) + ' MB'
    });
  }
}

// Call periodically to track memory usage
setInterval(monitorMemory, 5000);
```

Knowledge Check

- ▶ Q: What are three common causes of memory leaks in JavaScript?
- ▶ Q: How can you prevent memory leaks in JavaScript applications?

5. DOM Performance Optimization

Key Principles:

1. Minimize DOM Queries

```
// ❌ Multiple queries
function updateElements() {
  document.getElementById('title').textContent = 'New Title';
  document.getElementById('title').style.color = 'blue';
  document.getElementById('title').classList.add('highlight');
}

// ✅ Single query
function updateElements() {
  const titleElement = document.getElementById('title');
  titleElement.textContent = 'New Title';
  titleElement.style.color = 'blue';
  titleElement.classList.add('highlight');
}
```

2. Batch DOM Updates

```
// ❌ Multiple reflows
function addItem(items) {
```

```

const list = document.getElementById('list');
items.forEach(item => {
    const li = document.createElement('li');
    li.textContent = item;
    list.appendChild(li); // Triggers reflow each time
});
}

// ✅ Single reflow
function addItem(items) {
    const list = document.getElementById('list');
    const fragment = document.createDocumentFragment();

    items.forEach(item => {
        const li = document.createElement('li');
        li.textContent = item;
        fragment.appendChild(li);
    });

    list.appendChild(fragment); // Single reflow
}

```

3. Use Efficient Selectors

```

// ❌ Slow selectors
document.querySelectorAll('div.container > p.text');

// ✅ Fast selectors
document.getElementById('specificId');
document.getElementsByClassName('specificClass');

```

4. Avoid Layout Thrashing

```

// ❌ Layout thrashing
function animateElement() {
    const element = document.getElementById('box');
    element.style.left = '100px'; // Triggers layout
    const width = element.offsetWidth; // Forces layout calculation
    element.style.top = '100px'; // Triggers layout again
}

// ✅ Batch style changes
function animateElement() {
    const element = document.getElementById('box');
    element.style.cssText = 'left: 100px; top: 100px;'; // Single layout
}

```

5. Virtual Scrolling for Large Lists [29]

For lists with thousands of items, only render visible elements:

```
// Virtual scrolling implementation concept
function virtualScrolling(items, containerHeight, itemHeight) {
  const visibleItems = Math.ceil(containerHeight / itemHeight);
  const startIndex = Math.floor(scrollTop / itemHeight);
  const endIndex = startIndex + visibleItems;

  return items.slice(startIndex, endIndex);
}
```

Knowledge Check

- ▶ Q: Why is `document.createDocumentFragment()` useful for DOM performance?
- ▶ What causes DOM performance issues and how can you fix them?

6. Over-Optimization

Definition: Over-optimization occurs when you spend excessive time optimizing code that doesn't significantly impact performance.

Why Over-optimization is Problematic:

1. Premature Optimization Issues:

- **Wasted Time:** Optimizing non-critical code paths
- **Increased Complexity:** Making code harder to read and maintain
- **Reduced Flexibility:** Over-optimized code is harder to modify
- **Negligible Benefits:** 97% of optimizations have minimal impact

2. When NOT to Optimize:

- **Before profiling:** Never optimize without data
- **Non-critical paths:** Focus on actual bottlenecks
- **Readable code first:** Prioritize maintainability over micro-optimizations
- **Early development:** Optimize after core functionality is complete

Guidelines:

Measure First

```
// Use Performance API to measure
function measurePerformance(fn, name) {
  const start = performance.now();
  const result = fn();
  const end = performance.now();
  console.log(`${name} took ${end - start} milliseconds`);
  return result;
}

// Example usage
const result = measurePerformance(() => {
  return expensiveCalculation(1000);
}, 'Expensive Calculation');
```

80/20 Rule

- Focus on the 20% of code that causes 80% of performance issues
- Profile your application to identify bottlenecks
- Don't optimize code that runs infrequently

Premature Optimization

```
// ❌ Over-optimized for no reason
function addNumbers(a, b) {
  // Micro-optimization that doesn't matter
  return a + b >>> 0; // Unnecessary bitwise operation
}

// ✅ Clear and sufficient
function addNumbers(a, b) {
  return a + b;
}
```

When to Optimize:

- ✅ User-facing interactions feel slow
- ✅ Profiler shows clear bottlenecks
- ✅ Memory usage grows continuously
- ✅ App crashes or becomes unresponsive

When NOT to Optimize:

- ❌ Code runs once at startup
- ❌ Performance is already acceptable
- ❌ Optimization makes code unreadable

- ❌ No measurable improvement

Knowledge Check

- ▶ Q: What is the 80/20 rule in performance optimization?
- ▶ How do you avoid premature optimization while still writing performant code?

Interview Questions & Answers

- ▶ Q1: Explain the difference between debouncing and throttling with real-world examples.
- ▶ Q2: Implement a memoization function that can handle any number of arguments.
- ▶ Q: What is memoization and when should you use it?
- ▶ Q: How do you detect and prevent memory leaks?
- ▶ Q: What causes DOM performance issues and how do you fix them?
- ▶ Q: Explain the concept of over-optimization.
- ▶ Q3: What are memory leaks and how would you detect and prevent them?
- ▶ Q4: How would you optimize a function that processes large datasets?
- ▶ Q5: Explain how you would implement throttling for an API that has rate limits.

Practice Tasks

Task 1: Implement a debounced search function

Create a search input that only triggers after 500ms of no typing.

- ▶ 💡 Solution

Task 2: Create a throttled button click handler

Prevent button spam-clicking by allowing only one click per 2 seconds.

- ▶ 💡 Solution

Task 3: Implement memoization for Fibonacci sequence

Create a memoized version of the Fibonacci function.

► 💡 Solution

Task 4: Fix memory leak in event listener

Identify and fix the memory leak in this code:

```
function setupEventListener() {  
  const data = new Array(1000000).fill('large data');  
  
  document.getElementById('button').addEventListener('click', function() {  
    console.log('Button clicked');  
    console.log(data.length);  
  });  
}
```

► 💡 Solution

Task 5: Optimize DOM manipulation

Optimize this function that adds 1000 list items:

```
function addItem() {  
  const list = document.getElementById('list');  
  for (let i = 0; i < 1000; i++) {  
    const li = document.createElement('li');  
    li.textContent = `Item ${i}`;  
    list.appendChild(li); // Multiple reflows  
  }  
}
```

► 💡 Solution

Task 6: Create a performance monitoring utility

Build a simple utility to measure function execution time.

► 💡 Solution

Task 7: Implement a simple cache with TTL (Time To Live)

Create a caching system where entries expire after a specified time.

► 💡 Solution

Task 8: Create a rate limiter

Implement a rate limiter that allows only N requests per time window.

👥 Summary

Today you mastered crucial JavaScript performance optimization techniques:

- 🕒 **Debouncing**: Delay execution until activity stops
- 🚦 **Throttling**: Limit execution frequency
- 🧠 **Memoization**: Cache results to avoid recalculation
- 🐛 **Memory Leak Prevention**: Proper cleanup of resources
- 🚀 **DOM Optimization**: Efficient manipulation techniques
- ⚠️ **Smart Optimization**: When and when not to optimize

These techniques will make your JavaScript applications faster, more responsive, and more efficient!



Happy Coding! 🎉 Keep practicing and building amazing things with JavaScript!

🚀 *Let's Connect & Level Up Together*

Follow me for daily JavaScript tips, insightful notes, and project-based learning.

🔗 [X \(Twitter\)](#)

💼 [LinkedIn](#)

💬 [Telegram](#)

🧵 [Threads](#)

🔍 Dive into the full notes on GitHub → [40 Days JavaScript](#)

© 2025 • Crafted with ❤️ by Neeraj

📅 Next: Day - 37: JavaScript Web APIs Explained – Geolocation, Clipboard, Notifications, Playground, & More! 🥳 🥳 📅

🚀 Happy JavaScript coding! 🚀